

AIエージェントを作ろう!

Mastraを使ったAIエージェント作成

本日のアジェンダ

1. イントロダクション

- AIエージェントとMastraフレームワーク
- 本ハンズオンのゴール

2. 環境構築

- Mastraのインストールと設定

3. Mastraの基本

- "Hello, World!" エージェントの作成

4. 実践

- タスク実行AIエージェントを作ってみよう

5. まとめ

1. イントロダクション

自己紹介

- **名前:** (あなたの名前)
- **所属:** (あなたの所属)
- **経歴:** (あなたの経歴や専門分野など)
- **今日伝えたいこと:**
 - AIエージェント開発の楽しさと可能性
 - Mastraを使った開発の第一歩を踏み出すサポート

AIエージェントとは？

自律的にタスクを実行するAIプログラム

- **ゴール**を与えると、自ら**計画**を立てる
- 必要な**ツール**（Web検索、計算など）を使いこなす
- **状況を判断**し、ゴール達成まで自律的に動作する

Mastraは、このようなAIエージェントを効率的に開発するため

The image you are
requesting does not exist
or is no longer available.
imgur.com

Mastraフレームワークの紹介

TypeScript製のAIエージェント開発フレームワーク

- **特徴**

- **宣言的な構文**: エージェントの振舞いをシンプルに記述できる
- **ツールの統合**: 外部APIや自作関数を簡単にエージェントの「ツール」として追加可能
- **ワークフロー**: 複雑なタスクの実行フローを定義・自動化
- **開発体験**: プレイグラウンドやデバッグ機能が充実

本ハンズオンのゴール

- Mastraを使ったAIエージェント開発の**基本的な流れ**を理解する。
- 簡単な**ツール**を自作し、エージェントに組み込めるようになる。
- 自律的にタスクを実行する**AIエージェントを実際に動かし、その可能性を体感する。**

手を動かしながら、AIエージェント開発の第一歩を踏み出しましょう！

2. 環境構築

必要なもの

- **Node.js (v20以上)**
 - ターミナルで `node -v` を実行してバージョンを確認
- **テキストエディタ**
 - VS Codeを推奨します
- **OpenAI APIキー**
 - Platform.openai.com で取得
 - クレジットカード登録と利用料金にご注意ください

準備がまだの方は、この時間でセットアップをお願いします。

1. プロジェクトの準備

まず、作業用のディレクトリを作成し、`npm` プロジェクトを初期化します。

```
# 1. 作業ディレクトリを作成
mkdir mastra-handson
cd mastra-handson

# 2. npmプロジェクトを初期化
npm init -y
```

`package.json` というファイルが作成されればOKです。

2. Mastra と関連ライブラリのインストール

次に、MastraとTypeScript、そしてOpenAIのライブラリをインストールします。

```
# Mastra と OpenAIライブラリをインストール  
npm install mastra openai
```

```
# TypeScript関連ライブラリを開発用にインストール  
npm install -D typescript @types/node ts-node
```

tsconfig.json を作成します。

```
# tsconfig.json の雛形を作成
npx tsc --init
```

作成された `tsconfig.json` を以下のように編集します
(主要な部分のみ)。

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "strict": true,
```

4. APIキーの設定

プロジェクトのルートに `.env` ファイルを作成し、
OpenAIのAPIキーを記述します。

このファイルはGitなどで公開しないように注意してください。

.env

```
OPENAI_API_KEY="sk-..."
```

Mastraは自動でこのファイルを読み込んでくれます。

5. 動作確認

src ディレクトリを作成し、その中に **index.ts** を作成して、簡単なコードを書いてみましょう。

src/index.ts

```
console.log("Hello, Mastra!");
```

ターミナルで以下のコマンドを実行します。

```
npx ts-node src/index.ts
```

Hello, Mastra! と表示されれば、環境構築は完了です！

3. Mastraの基本

Mastraの主要コンセプト

Mastraは主に3つの要素で構成されます。

1. Tool (ツール)

- エージェントが利用できる具体的な「能力」。
- 例: Web検索する, 計算する, メールを送る
- 実体はただのTypeScriptの関数です。

2. Agent (エージェント)

- Tool を使いこなし、与えられた指示を実行する思考エンジン。
- どの Tool をどの順番で使うかをLLMが判断します。

3. Workflow (ワークフロー)

最初のAIエージェント：Hello, World!

ユーザーの入力をそのままオウム返しする、最もシンプルなエージェントを作ってみましょう。

`src/index.ts` を以下のように書き換えます。

```
import { Agent, tool, workflow } from 'mastra';
import { OpenAI } from 'openai';

// 1. ツールの定義
const echo = tool({
  name: 'echo',
  description: 'Echo the input back to the user.',
  input: { text: 'The text to echo' },
  run: async ({ input }) => {
    return input.text;
  },
});

// 2. エージェントの定義
const echoAgent = new Agent({
  llm: new OpenAI(),
  tools: [echo],
  prompt: 'You are an echo agent. Just use the echo tool with the user input.',
});

// 3. ワークフローの定義
const main = workflow({
  {
    name: 'main',
```

Hello, World! の実行

ターミナルで実行してみましょう。

```
npx ts-node src/index.ts
```

実行結果

```
{  
  ok: true,  
  output: 'こんにちは、Mastra!',  
  events: [ ... ]  
}
```

output にインプットと同じ文字列が返ってきたら成功です！

コードの解説 (1/3) - Tool

```
// ただのTypeScript関数に説明を付け加えたもの
const echo = tool({
  // ツールの名前 (エージェントがどのツールを使うか判断するのに使う)
  name: 'echo',
  // ツールの説明 (エージェントがツールの役割を理解するのに使う)
  description: 'Echo the input back to the user.',
  // このツールが受け取る引数の定義
  input: { text: 'The text to echo' },
  // 実際の処理
  run: async ({ input }) => {
    return input.text;
  },
});
```

`tool` 関数でラップすることで、ただの関数をAIエージェントが理解できる「ツール」に変換します。 `name` と

コードの解説 (2/3) - Agent

```
const echoAgent = new Agent({  
  // 使用するLLMのインスタンス  
  llm: new OpenAI(),  
  // エージェントが使用可能なツールのリスト  
  tools: [echo],  
  // エージェントの役割や振る舞いを指示するプロンプト  
  prompt: 'You are an echo agent. Just use the echo tool with the user input.',  
});
```

Agent は思考を担当します。どの **tool** をどの引数で呼び出すかを、**prompt** と **tools** の **description** をヒントにLLMが判断します。

コードの解説 (3/3) - Workflow

```
const main = workflow(  
  {  
    name: 'main', // ワークフローの名前  
    agent: echoAgent, // このワークフローで使うエージェント  
  },  
  async ({ input, agent }) => {  
    // ワークフローの具体的な処理  
    return await agent.run({ input });  
  }  
);  
  
// ワークフローを実行  
main({ input: 'こんにちは、Mastra!' }).then(console.log);
```

workflow は処理全体の流れを定義します。ユーザーからの入力を受け取り、**agent** を呼び出し、結果を返すエン

4. 実践：タスク実行AIエージェントの作成

作るもの

「今日の東京の天気を調べて、服装を提案してくれるエージェント」

- **入力:** "今日の東京の天気と服装を教えて"
- **エージェントの思考プロセス:**
 1. 今日が何月何日か調べる必要があるな
 2. 「X月X日 東京の天気」でWeb検索しよう
 3. 天気と気温が分かったから、それに基づいて服装を考えよう
 4. 結果をまとめてユーザーに返そう
- **出力:** "今日の東京の天気は晴れ、最高気温は28℃です。"

Step 1: エージェントの設計

このエージェントには、少なくとも3つの能力（ツール）が必要そうです。

1. **getCurrentDate** : 今日の日付を取得するツール
2. **searchWeb** : 与えられたクエリでWeb検索するツール
3. **suggestClothing** : 天気と気温から服装を提案するツール

これらのツールを準備し、それらを使いこなせるエージェントとワークフローを構築していきます。

Step 2: ツールの実装 (1/3)

getCurrentDate

まずは今日の日付を返す簡単なツールから作りましょう。
`src/tools.ts` というファイルを新しく作成します。

```
// src/tools.ts
import { tool } from 'mastra';

export const getCurrentDate = tool({
  name: 'getCurrentDate',
  description: 'Get the current date in YYYY-MM-DD format.',
  run: async () => {
    return new Date().toLocaleDateString('ja-JP', {
      year: 'numeric',
      month: '2-digit',
      day: '2-digit',
    });
  }
});
```

Step 2: ツールの実装 (2/3) **searchWeb**

次にWeb検索ツールです。

今回は本物の検索APIは使わず、特定のクエリに対してダミーの検索結果を返す**モック（偽物）**として実装します。

`src/tools.ts` に追記します。

```
// src/tools.ts

// ... (getCurrentDate の下に追記)

export const searchWeb = tool({
  name: 'searchWeb',
  description: 'Search the web for the given query.',
  input: { query: 'The search query' },
  run: async ({ input }) => {
```

Step 2: ツールの実装 (3/3)

suggestClothing

最後に、天気と気温の情報から服装を提案するツールです。

このツールはLLMの推論能力に頼るため、内部で再度LLMを呼び出す構成にします。

`src/tools.ts` に追記します。

```
// src/tools.ts
import { OpenAI } from 'openai';
import { tool } from 'mastra';

// ... (searchWeb の下に追記)

export const suggestClothing = tool({
  name: 'suggestClothing',
  description: 'Suggest clothing based on the weather and temperature.',
  inputs: {
```

Step 3: エージェントとワークフローの構築

3つのツールができたので、これらを使ってエージェントとワークフローを構築します。

`src/index.ts` を以下のように全て書き換えます。

```
// src/index.ts
import { Agent, workflow } from 'mastra';
import { OpenAI } from 'openai';
import { getCurrentDate, searchWeb, suggestClothing } from './tools';

const weatherAgent = new Agent({
  llm: new OpenAI({ model: 'gpt-4o-mini' }),
  tools: [getCurrentDate, searchWeb, suggestClothing],
  prompt: `
    You are a helpful weather assistant.
    Your goal is to provide the weather forecast and suggest appropriate clothing.
    1. First, get the current date.
    2. Second, search the web for the weather in Tokyo for that date.
    3. Finally, suggest clothing based on the weather information.
    4. Provide the final answer to the user in Japanese.
  `,
});

const main = workflow(
  {
    name: 'weatherWorkflow',
    agent: weatherAgent,
  },
  ©2025 nextbeat Co., Ltd.
```

Step 4: 実行と結果の確認

それでは、完成したエージェントを実行してみましょう！

```
npx ts-node src/index.ts
```

実行ログの例

[Web Search] Searching for: 2025-08-29 東京の天気

--- Final Output ---

今日の東京の天気は晴れ、最高気温は28℃です。この気温であれば、半袖のTシャツやポロシャツに、薄手のパンツやスカートを合わせるのがおすすめです。快適に過ごせるでしょう。

エージェントが自らツールを呼び出し、最終的な答えを導き出している様子がわかります。

デバッグとログの活用

Mastraは実行過程をイベントとして記録しており、デバッグに役立ちます。

`agent.run()` の返り値には `events` 配列が含まれています。

```
main({ input: '...' }).then((res) => {
  if (res.ok) {
    console.log('--- Final Output ---');
    console.log(res.output);
    console.log('\n--- Events ---');
    // どのような思考プロセスだったかを確認
    res.events.forEach((e, i) => {
      if (e.type === 'tool_call') {
        console.log(`[${i}] Tool Call: ${e.name}(${JSON.stringify(e.input)})`);
      }
      if (e.type === 'tool_output') {
        console.log(`[${i}] Tool Output: ${JSON.stringify(e.output)}`);
      }
    });
  }
});
```

ハンズオン：カスタマイズしてみよう！

いくつかカスタマイズのアイデアを提案します。ぜひ挑戦してみてください。

- **アイデア1: 新しいツールを追加する**
 - `getLuckyColor` のような、今日のラッキーカラーを返すツールを追加し、服装提案に含めてもらう。
- **アイデア2: プロンプトを修正する**
 - エージェントの性格を変えてみる（例：ギャル風、執事風）。
 - `weatherAgent` の `prompt` を修正してみましよう。
- **アイデア3: 別の都市の天気を調べる**

5. まとめ

発展的なトピック

Mastraには、今日紹介した機能以外にも高度な機能があります。

- **Memory (メモリ)**

- 過去の対話履歴を記憶し、文脈に沿った応答を可能にする機能。チャットボットなどで活躍します。

- **RAG (Retrieval-Augmented Generation)**

- 外部のドキュメントやデータベースの情報を検索し、その内容に基づいて回答を生成する仕組み。社内文書に基づいたQAボットなどが作れます。

- **Playground (プレイグラウンド)**

参考資料と次のステップ

- **Mastra 公式ドキュメント**

- <https://mastra.io/docs> (*URLはダミーです。
公式サイトをご確認ください)
- 全ての機能について詳細な解説があります。

- **Mastra GitHubリポジトリ**

- <https://github.com/mastra-dev/mastra>
(*URLはダミーです)
- サンプルコードや最新の動向はこちらで。

- **今日作ったコードを発展させる**

- ダミーだった [searchWeb](#) を 実際の検索API

質疑応答

ご清聴ありがとうございました！